



MODUL 10 · VALIDASI DATA & FORMS

React 19 Form Hooks: useActionState & useFormStatus

Native React 19 approach — form yang bekerja dengan dan tanpa JavaScript (progressive enhancement).



useActionState — Pengganti useFormState

Dari 'react' (bukan react-dom!). Menghubungkan Server Action ke form state. isPending built-in.

```
"use server"
import { z } from "zod"

const contactSchema = z.object({
  email: z.string().email("Email tidak valid"),
  message: z.string().min(10, "Min 10 karakter"),
})

// ✅ Signature: (prevState, formData)
// prevState = state sebelumnya
export async function submitContact(
  prevState: {
    errors?: Record<string, string[]>
    success?: boolean
  },
  formData: FormData
) {
  const raw = {
    email: formData.get("email"),
    message: formData.get("message"),
  }
  const validated = contactSchema.safeParse(raw)
  if (!validated.success) {
    return {
      errors:
        validated.error.flatten().fieldErrors,
    }
  }
  await sendEmail(validated.data)
  return { success: true }
}
```

```
'use client'
// ✅ from 'react' — BUKAN 'react-dom'!
import { useActionState } from 'react'
import { submitContact } from '@actions/contact'

export function ContactForm() {
  const [state, formAction, isPending] = useActionState(submitContact,
    {
      errors: undefined,
      success: false,
    })

  if (state.success) {
    return <p>✅ Pesan terkirim!</p>
  }

  return (
    <form action={formAction}>
      <input name="email" type="email" />
      {state.errors?.email?.[0] && (
        <p className="text-red-500">{state.errors.email[0]}</p>
      )}
      <textarea name="message" />
      {state.errors?.message?.[0] && (
        <p className="text-red-500">{state.errors.message[0]}</p>
      )}
      <button disabled={isPending}>
        {isPending ? 'Mengirim...' : 'Kirim'}
      </button>
    </form>
  )
}
```

useFormStatus — Submit Button Aware

Baca pending state dari form parent. WAJIB di child component terpisah.

```
// ✅ Import dari 'react-dom' (tetap react-dom, tidak berubah)
import { useFormStatus } from 'react-dom'

// WAJIB: komponen terpisah yang ada DI DALAM <form>
// useFormStatus membaca state dari PARENT form
function SubmitButton({ label = 'Simpan' }: { label?: string }) {
  const { pending } = useFormStatus()
  // React 19 juga: data (FormData), method, action

  return (
    <button
      type="submit"
      disabled={pending}
      className={pending ? 'opacity-50 cursor-not-allowed' : ''}
    >
      {pending ? 'Memproses...' : label}
    </button>
  )
}

// ✅ Pemakaian — SubmitButton di dalam <form>:
export function ContactForm() {
  const [state, formAction] = useActionState(submitContact, {})
  return (
    <form action={formAction}>
      <input name="email" />
      <SubmitButton label="Kirim Pesan" /> {/* ✅ child dari form */}
    </form>
  )
}

// ❌ JANGAN — useFormStatus di komponen yang SAMA dengan form:
function ContactForm() {
  const { pending } = useFormStatus() // ← TIDAK bekerja!
}
```

Kapan Pakai Pendekatan Mana?

Tiga pendekatan — pilih berdasarkan kompleksitas dan kebutuhan.

useActionState saja

Cocok untuk: Form sederhana: contact, newsletter, simple CRUD

✓ Minimal code. Progressive enhancement. Tidak perlu JS. ⚠ Tidak ada real-time validation. UX kurang responsif.

react-hook-form + Zod

Cocok untuk: Form kompleks client-only (dashboard, SPA)

✓ Real-time validation. UX excellent. Field control penuh. ⚠ Tidak progressive enhancement. Butuh JS selalu.

react-hook-form + useActionState

Cocok untuk: Form production: register, create post, edit profile

✓ Real-time + server validation. Progressive enhancement. Best UX. ⚠ Lebih banyak setup code. useEffect untuk sync errors.

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — useState diimport dari mana di React 19?

A) from 'react' — bukan react-dom!

B) from 'react-dom' (seperti useFormStatus)

C) from 'next/form'

Q2 — Kenapa useFormStatus harus di child component terpisah?

A) useFormStatus membaca context dari PARENT form — harus child/nested dari form

B) Best practice naming saja

C) Masalah TypeScript typing

Q3 — Signature Server Action berubah saat dipakai dengan useState. Apa yang berubah?

A) Menerima prevState sebagai arg pertama: (prevState, formData) bukan (formData)

B) Tidak ada yang berubah

C) Harus return void

Tugas Mandiri

Implementasikan form dengan React 19 hooks.

01 Contact Form

Buat ContactForm dengan useActionState saja. Server Action return {errors} atau {success:true}. Tampilkan state.errors per field.

02 SubmitButton

Ekstrak SubmitButton ke komponen terpisah. Gunakan useFormStatus dari react-dom. Tampilkan spinner saat pending. Gunakan di semua form.

03 Register Form

Buat RegisterForm dengan KOMBINASI: react-hook-form (client) + useActionState (server). useEffect sync server errors ke form fields.

04 Test Progressive

Disable JavaScript di browser (DevTools → Settings → Disable JavaScript). Submit form → server action tetap berjalan. Enable kembali → UX normal.

⚠ useActionState dari 'react'. useFormStatus dari 'react-dom'. Import path berbeda — jangan tertukar!

Yang Sudah Kita Pelajari

- ✓ `useActionState`: dari 'react' (bukan react-dom!). Signature SA berubah: (prevState, formData).
- ✓ `useFormStatus`: dari 'react-dom'. Harus di child/nested component dari `<form>`.
- ✓ Progressive enhancement: `<form action={serverAction}>` bekerja tanpa JavaScript.
- ✓ Kombinasi terbaik: react-hook-form (real-time) + `useActionState` (server) + `useEffect sync`.
- ✓ `SubmitButton`: komponen terpisah dengan `useFormStatus` untuk pending state yang clean.

→ Selanjutnya: **Modul 12 — API Routes & Server Actions**

